CloudMart AWS Deployment Runbook

This document records the current CloudMart AWS setup, what has already been implemented, how to deploy it, and what remains for the final production-grade Kubernetes submission.

Current Goal

The immediate goal is a low-cost AWS deployment path:

```
GitHub Actions -> ECR -> ECS Fargate -> CloudMart frontend + backend containers
```

This is a temporary AWS-native deployment path before the final EKS Kubernetes deployment.

Important assignment note:

```
ECS does not replace the final EKS / managed Kubernetes requirement.
```

ECS is being used now to get the app running on AWS earlier and at lower complexity. The final assignment still needs EKS, Kubernetes manifests, Ingress, HPA, NetworkPolicies, and related controls.

What Has Been Done

11-Stage Progress Summary

This is the simple stage-by-stage view of the full project.

Stage	Status	What It Means	What To Do
Stage 0: Repository Structure	Completed	Project folders, docs folders, infrastructure folders, GitHub workflow folder, and service folders are prepared.	Keep source code organized in the existing structure.

Stage	Status	What It Means	What To Do
Stage 1: Terraform Backend	Completed	S3, DynamoDB, and KMS were created for remote Terraform state.	Use this backend for staging/prod Terraform. Do not delete it until all environments are destroyed.
Stage 2: Networking	Code ready, apply pending	Terraform code exists for VPC, subnets, routes, security groups, and endpoints.	Apply staging Terraform first. Keep NAT Gateway and Flow Logs disabled for low cost.
Stage 3: Container Registry	Code ready, apply pending	Terraform code exists for five ECR repositories with scan-on-push and lifecycle cleanup.	Apply staging Terraform, then GitHub Actions can push images.
Stage 4: Managed Backends	Not started	DynamoDB, SQS, RDS PostgreSQL, SES, Secrets Manager, and KMS for the app are not fully added yet.	Add DynamoDB and SQS first because they are low-cost. Add RDS later because it costs more.
Stage 5: ECS Temporary Deployment	Code ready, apply pending	ECS is added as a temporary AWS deployment target before EKS.	Apply staging Terraform, add GitHub secrets, then run the ECS workflow.
Stage 6: EKS Kubernetes Cluster	Not started	Final managed Kubernetes cluster is not created yet.	Add EKS Terraform after ECS demo works.
Stage 7: Kubernetes App Manifests	Not started	Namespaces, Deployments, Services, Ingress, HPA, PDB, ConfigMaps, Secrets, and NetworkPolicies are still pending.	Create Kubernetes manifests after EKS is ready.
Stage 8: CI/CD Final Pipeline	Partially done	GitHub Actions exists for Terraform checks, service CI, ECR push, and ECS deploy.	Later extend workflow to deploy to EKS staging/prod.

Stage	Status	What It Means	What To Do
		Kubernetes deploy is not done yet.	
Stage 9: Observability and Security	Not started	CloudWatch dashboards, alarms, GuardDuty, WAF, IRSA, and policy controls are pending.	Add after app runs on EKS.
Stage 10: Cost Management	Partially done	Tags and low-cost defaults are added, but budgets and cost reports are not done.	Add AWS Budget, collect Cost Explorer screenshots, calculate cost per 1,000 orders.
Stage 11: Disaster Recovery and Final Deliverables	Not started	Backups, restore proof, diagrams, report, ADR summaries, and demo script are pending.	Complete near the end after infrastructure is stable.

Current practical next step:

```
Apply Stage 2 + Stage 3 + Stage 5 in staging.
```

That means applying the staging Terraform environment, which now creates:

```
VPC networking
ECR repositories
ECS temporary deployment foundation
```

Then run GitHub Actions to build and deploy the app to ECS.

Stage 0: Repository Structure

The repository now contains:

```
services/       Application source code for all 5 services
infra/          Terraform infrastructure code
k8s/            Placeholder for future Kubernetes manifests
ecs/            ECS task definition template
docs/           ADRs and setup documentation
.github/        GitHub Actions workflows
```

Application services:

```
product-service
order-service
user-service
notification-service
frontend
```

Stage 1: Terraform Backend Bootstrap

Terraform backend resources were created in AWS:

```
S3 bucket:       cloudmart-13-tfstate-804431973197
DynamoDB table: cloudmart-13-terraform-locks
KMS key:         arn:aws:kms:ap-south-1:804431973197:key/42d91158-a8a7-407c-b57c-43c1433e3f07
```

These are used to store and lock Terraform state remotely.

Files:

```
infra/bootstrap/main.tf
infra/bootstrap/variables.tf
infra/bootstrap/outputs.tf
infra/bootstrap/terraform.tfvars.example
```

Stage 2: Low-Cost Networking

Terraform network module has been created.

It creates:

```
VPC
2 Availability Zones
public subnets
private app subnets
private data subnets
Internet Gateway
route tables
security groups for ALB, EKS nodes, RDS, bastion
S3 gateway VPC endpoint
DynamoDB gateway VPC endpoint
optional NAT Gateway
optional VPC Flow Logs
```

Low-cost defaults:

```
enable_nat_gateway = false
enable_flow_logs   = false
```

This avoids NAT Gateway hourly cost and CloudWatch Flow Log ingestion cost during early testing.

Files:

```
infra/modules/network/
infra/envs/staging/
infra/envs/prod/
```

Stage 3: ECR Repositories

Terraform ECR module has been created.

It creates repositories for:

```
cloudmart/product-service
cloudmart/order-service
cloudmart/user-service
cloudmart/notification-service
cloudmart/frontend
```

Each repository has:

```
scan on push enabled
lifecycle policy keeping last 10 images
```

Files:

```
infra/modules/ecr/
```

Temporary Stage: ECS Fargate Deployment

Terraform ECS module has been created.

It creates:

```
ECS cluster
ECS Fargate service
ECS task execution role
ECS task role
ECS security group allowing HTTP port 80
CloudWatch log group
```

The ECS service starts with:

```
ecs_desired_count = 0
```

This means no Fargate task runs until GitHub Actions deploys and scales the service to `1`.

This keeps idle cost low.

Files:

```
infra/modules/ecs/
ecs/task-definition.json
services/frontend/nginx.ecs.conf
```

GitHub Actions

Workflows created:

```
.github/workflows/terraform-bootstrap.yml
.github/workflows/terraform-checks.yml
.github/workflows/terraform-environment.yml
.github/workflows/services-ci.yml
.github/workflows/ecs-build-push-deploy.yml
```

Main ECS workflow:

```
ECS Build Push Deploy
```

It performs:

```
build all 5 Docker images
scan each image with Trivy
push images to ECR
register ECS task definition
update ECS service
wait until ECS service is stable
```

The frontend ECS image uses:

```
services/frontend/nginx.ecs.conf
```

This proxies backend calls to localhost because all containers run inside the same ECS Fargate task.

Local Tool Setup

Required local tools:

```
Terraform
AWS CLI
Git
Docker Desktop, optional for local app testing
```

Check Terraform:

```
terraform version
```

Check AWS CLI:

```
aws --version
```

Check AWS account:

```
aws sts get-caller-identity
```

Local AWS Credential Setup

For local Terraform testing and apply:

```
aws configure
```

Use:

```
AWS Access Key ID
AWS Secret Access Key
Default region: ap-south-1
Default output: json
```

Verify:

```
aws sts get-caller-identity
```

Make sure the returned account ID is the correct AWS account before running Terraform.

Stage 1 Bootstrap Commands

Already completed, but these are the commands used:

```
cd /d "D:\academics\L4-S2\cloud management\Cloudmart\infra\bootstrap"
copy terraform.tfvars.example terraform.tfvars
notepad terraform.tfvars
terraform init
terraform validate
terraform plan -var-file=terraform.tfvars
terraform apply -var-file=terraform.tfvars
```

Expected outputs:

```
terraform_state_bucket
terraform_lock_table
terraform_state_kms_key_arn
```

Deploy Staging Infrastructure

Run from CMD:

```
cd /d "D:\academics\L4-S2\cloud management\Cloudmart\infra\envs\staging"
copy terraform.tfvars.example terraform.tfvars
notepad terraform.tfvars
```

Recommended values:

```
aws_region  = "ap-south-1"
project     = "cloudmart"
environment = "staging"
team_id     = "13"
owner_email = "yasiram447@gmail.com"

vpc_cidr                 = "10.0.0.0/16"
az_count                 = 2
enable_nat_gateway       = false
enable_gateway_endpoints = true
enable_flow_logs         = false

ecs_allowed_http_cidrs        = ["0.0.0.0/0"]
ecs_task_cpu                  = "1024"
ecs_task_memory               = "2048"
ecs_desired_count             = 0
ecs_log_retention_days        = 7
ecs_enable_container_insights = false
```

Initialize with the remote backend:

```
terraform init ^
  -backend-config="bucket=cloudmart-13-tfstate-804431973197" ^
  -backend-config="key=staging/terraform.tfstate" ^
  -backend-config="region=ap-south-1" ^
  -backend-config="dynamodb_table=cloudmart-13-terraform-locks" ^
  -backend-config="encrypt=true"
```

Validate:

```
terraform validate
```

Preview:

```
terraform plan -var-file=terraform.tfvars
```

Apply:

```
terraform apply -var-file=terraform.tfvars
```

After apply:

```
terraform output
```

GitHub Secrets Required

Add secrets in:

```
GitHub repository -> Settings -> Secrets and variables -> Actions
```

Required for AWS access:

```
AWS_ROLE_TO_ASSUME
```

Example:

```
arn:aws:iam::804431973197:role/cloudmart-github-actions
```

Required for ECS deployment:

```
ECS_CLUSTER_NAME
ECS_SERVICE_NAME
ECS_TASK_EXECUTION_ROLE_ARN
ECS_TASK_ROLE_ARN
JWT_SECRET
```

Get these from:

```
terraform output
```

Mapping:

```
ecs_cluster_name              -> ECS_CLUSTER_NAME
ecs_service_name              -> ECS_SERVICE_NAME
ecs_task_execution_role_arn   -> ECS_TASK_EXECUTION_ROLE_ARN
ecs_task_role_arn             -> ECS_TASK_ROLE_ARN
```

Use a long random value for:

```
JWT_SECRET
```

GitHub OIDC Role

GitHub Actions should use AWS OIDC, not long-lived AWS access keys.

Create an IAM OIDC provider:

```
https://token.actions.githubusercontent.com
```

Audience:

```
sts.amazonaws.com
```

Create an IAM role trusted by the GitHub repository.

The GitHub role needs permissions for:

```
ECR image push
ECS task definition registration
ECS service update
CloudWatch log group read/create
iam:PassRole for ECS task roles
```

Deploy Application With GitHub Actions

Go to:

```
GitHub -> Actions -> ECS Build Push Deploy -> Run workflow
```

Choose:

```
environment: staging
deploy_to_ecs: true
```

The workflow will:

```
build images
scan with Trivy
push to ECR
register ECS task definition
update ECS service
scale ECS desired count to 1
```

Open The App

Because the temporary ECS setup has no load balancer, use the public IP of the running ECS task.

List ECS tasks:

```
aws ecs list-tasks ^
  --cluster <ECS_CLUSTER_NAME> ^
  --service-name <ECS_SERVICE_NAME> ^
  --region ap-south-1
```

Describe task:

```
aws ecs describe-tasks ^
  --cluster <ECS_CLUSTER_NAME> ^
  --tasks <TASK_ARN> ^
  --region ap-south-1
```

Find the task network interface ID from the output, then:

```
aws ec2 describe-network-interfaces ^
  --network-interface-ids <ENI_ID> ^
  --query "NetworkInterfaces[0].Association.PublicIp" ^
  --output text ^
  --region ap-south-1
```

Open:

```
http://<PUBLIC_IP>
```

Health check:

```
http://<PUBLIC_IP>/health
```

Expected:

```
{"status":"healthy","service":"frontend","platform":"ecs"}
```

Stop ECS Runtime Cost

After testing, stop the running Fargate task:

```
aws ecs update-service ^
  --cluster <ECS_CLUSTER_NAME> ^
  --service <ECS_SERVICE_NAME> ^
  --desired-count 0 ^
  --region ap-south-1
```

This keeps the ECS service but stops the running task.

Destroy Staging Infrastructure

Only do this when you want to remove staging resources:

```
cd /d "D:\academics\L4-S2\cloud management\Cloudmart\infra\envs\staging"
terraform destroy -var-file=terraform.tfvars
```

Review the destroy plan before typing:

```
yes
```

This removes staging VPC, ECR, ECS, IAM roles, security groups, and logs managed by this Terraform environment.

Destroy Bootstrap Infrastructure

Only do this after all environment Terraform states are no longer needed:

```
cd /d "D:\academics\L4-S2\cloud management\Cloudmart\infra\bootstrap"
terraform destroy -var-file=terraform.tfvars
```

This removes:

```
S3 Terraform state bucket
DynamoDB lock table
KMS key alias
KMS key scheduled for deletion
```

AWS KMS keys are not deleted instantly. AWS schedules deletion after the waiting period.

What Is Still Remaining

Required For Final Assignment

The final assignment requires managed Kubernetes. Remaining major work:

```
EKS cluster
EKS managed node group
Kubernetes namespaces
Kubernetes Deployments
Kubernetes Services
Ingress with AWS Load Balancer Controller
HPA for product-service and order-service
NetworkPolicies
IRSA per service
Secrets Manager integration
RDS PostgreSQL for user-service
DynamoDB for product-service
SQS for order events
SES for email notifications
CloudWatch monitoring and alerts
GuardDuty
WAF
Cost budget
Disaster recovery plan
Velero or Git-based manifest backup
Architecture diagrams
Final report
Live demo script
```

Recommended Next Implementation Order

```
1. Apply staging Terraform with network + ECR + ECS
2. Add GitHub secrets
3. Run ECS Build Push Deploy
4. Confirm app opens through ECS task public IP
5. Stop ECS service desired count to 0 after testing
6. Add managed AWS backends: DynamoDB, SQS, Secrets Manager
7. Add RDS only when needed because it has ongoing cost
8. Add EKS Terraform
9. Move deployment from ECS to Kubernetes
10. Add observability, security hardening, cost management, DR evidence
```

Cost Notes

Low-cost settings currently used:

```
NAT Gateway disabled
VPC Flow Logs disabled
ECS desired count starts at 0
ECS Container Insights disabled
CloudWatch log retention set to 7 days
ECR lifecycle policy keeps only last 10 images
```

Resources that may still cost money:

```
KMS key
ECR storage
CloudWatch logs
Fargate task when desired count is 1
public IPv4 usage, depending on AWS pricing
```

More expensive future resources:

```
NAT Gateway
RDS PostgreSQL
EKS control plane
EKS worker nodes
Application Load Balancer
```

Current Deployment Architecture

Temporary ECS architecture:

```
Browser
  -> ECS Fargate Task Public IP :80
    -> frontend container
      -> /api/products -> product-service on 127.0.0.1:8001
      -> /api/orders   -> order-service on 127.0.0.1:8002
      -> /api/auth     -> user-service on 127.0.0.1:8003
      -> notification-service runs internally
```

All services currently use in-memory backends for low-cost testing.

Final target architecture:

```
Browser
  -> ALB / Ingress
    -> frontend
    -> product-service -> DynamoDB
    -> order-service -> SQS + product-service
    -> user-service -> RDS PostgreSQL
    -> notification-service -> SQS + SES
```